Day-19 28-Feb-2020

Async Javascript

19-promises.js

// Three state of Promises
Pending, done (fulfil, resolve),
nop (reject, reject, notice)

```
Const promise = new Promise()
Console.log (promise)
```

↓

Resolve not called
↳ It is just callback (fancy name resolver)

Event loop का काम
Call stack में
क्या जाएगा

→ Event loop is a while loop
→ Event loop manage करता है

```
Const promise = new Promise ((res, rej) => {
        Set Timeout (() => {
            res ("Chai Code")
        } ,2000)
    })

Console.log (promise)

setTimeout (() => {
    Console.log (promise);
} , 3000)
```

↳ bad practice we not know how much time take to solve promises (situation like db connection

Promise.then((value) => {
    Console.log(value)
});

↓
Chrome

जो भी resolve होती है then के callback में as it as pass हो जाती है।

Note

Promise.then(console.log);

↓
Chrome

but यहाँ पर हमने callback pass नहीं किया तो log होने लगा क्योंकि हमने callback ही तो pass करवाना और then automatic value उस callback में पास कर देगा

console.log में भी तो function होता है तो log में automatic then value pass कर दिया काम हो गया।

reject में भी value पास तो करना है पर resolve जब होगी reject क्यों। Proper error पता करने के लिए किया जाता है।

then भी reject solve करता है

वह then के 2nd Parameter में लिखा जाता है

```
Promise.then(
    (data) => Console.log(data),
    (error) => Console.log(error)
)
```

<u>Or</u>

```
Promise.then(
        (data) => Console.log(data))
        .Catch((error) => Console.log(error));
```

(*)

```
Const Promise = new Promise((res, rej) => {
        SetTimeout(() => {
            res("Chai Code");
            rej(new
```

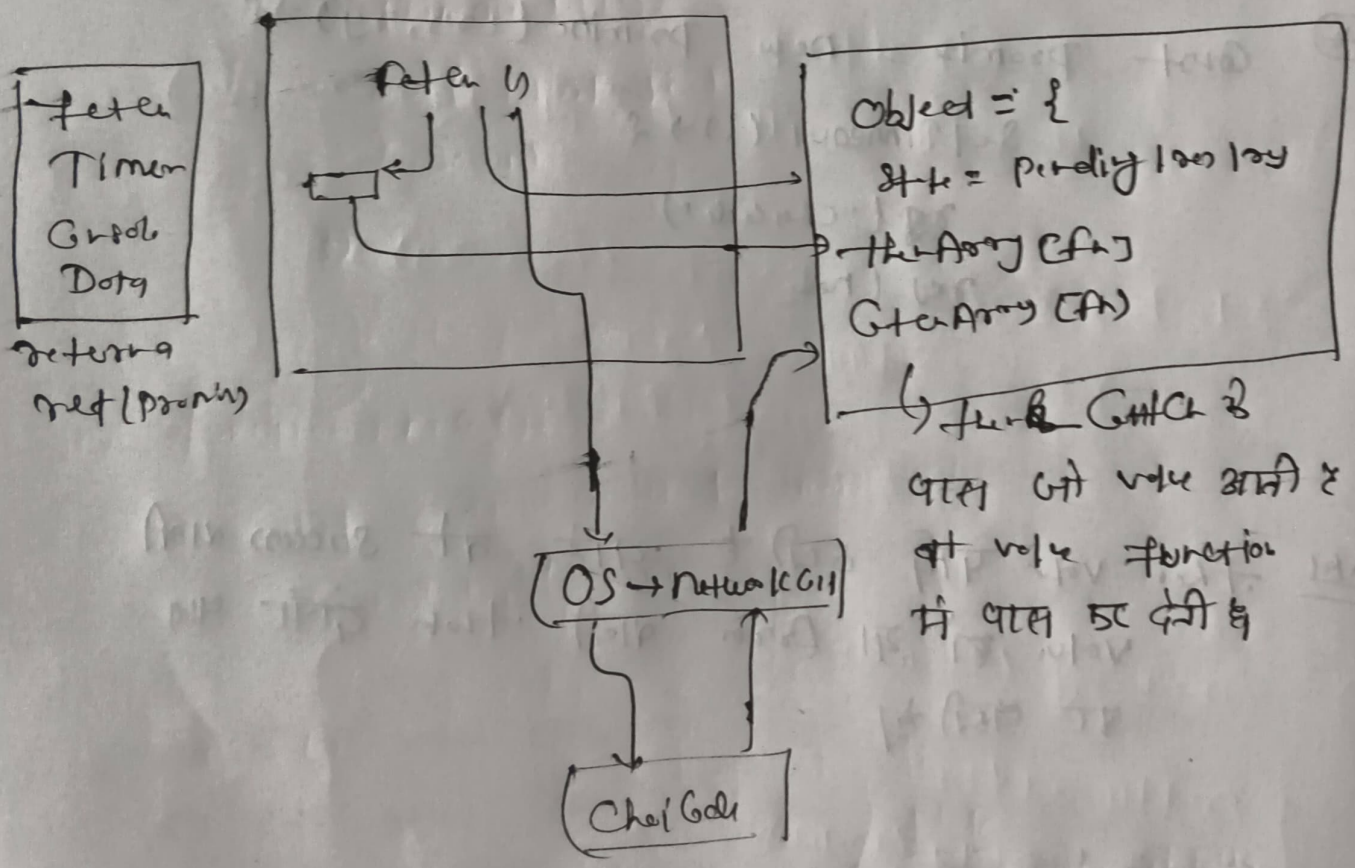
<u>Note</u> then value पाए कहीं है चाहे तो success वाली value हो या Error वाली then दोनों पाता है कर सकती है।

```
Promise
.then((data) => {
        newData = data.toUpperCase();
        return newData;
    })
.then((data) => {
        return data + ".com";
    })
.then(Console.log)
```

.Catch (error) {
Console.log (error)
}

.then . fetch की
limit है 45K
then . Catch then . Cat
45K

* सबसे ज्यादा Promise fetch के साथ होता है
→ fetch Promise return करती है
→ fetch का तब काम return a reference (Promise)



then Catch के पास जो value आती है वो value function में पास कर देती है

Const tusort = Promise.resolve("Tusort");

Grabobing (tusort):

↳ If your role to immediate Promise Resolve then the syntax is okay

Const allpromise = Promise.any([
Promise.resolve("Choi"),
Promise.resolve("Code")
])

allpromise.then(Grabobing);
↓
Choi

Const allpromise = promise.any([
promise.resolve("choi");
promise.resolve("Code");
promise.reject("Error");
])

~~Grabobing~~
allpromise.then(Grabobing)
↓
Choi

Const allpromise =

Assignment
promise & all
method
Blog

```
Const promise = new promise((res, rej) =>{
    setTimeout(() =>{
      res("मुकेश");
      }, 3000);
    });

async function nice(){
    Const result = await promise;
    Console.log(result)
  }
  nice();

Const result = await promise;

~~result~~;
Console.log(result);
```

Note
अब आप globally
await use कर
सकते हो।

await, async में error handling नहीं हो पाता

so use try & catch

```
async function Nia (){
    try{
        const result = await promise;
        console.log (result);
    } catch (error){
        Console.log ("Error aa gya ji", error.)
    }
```

20 - promise.js

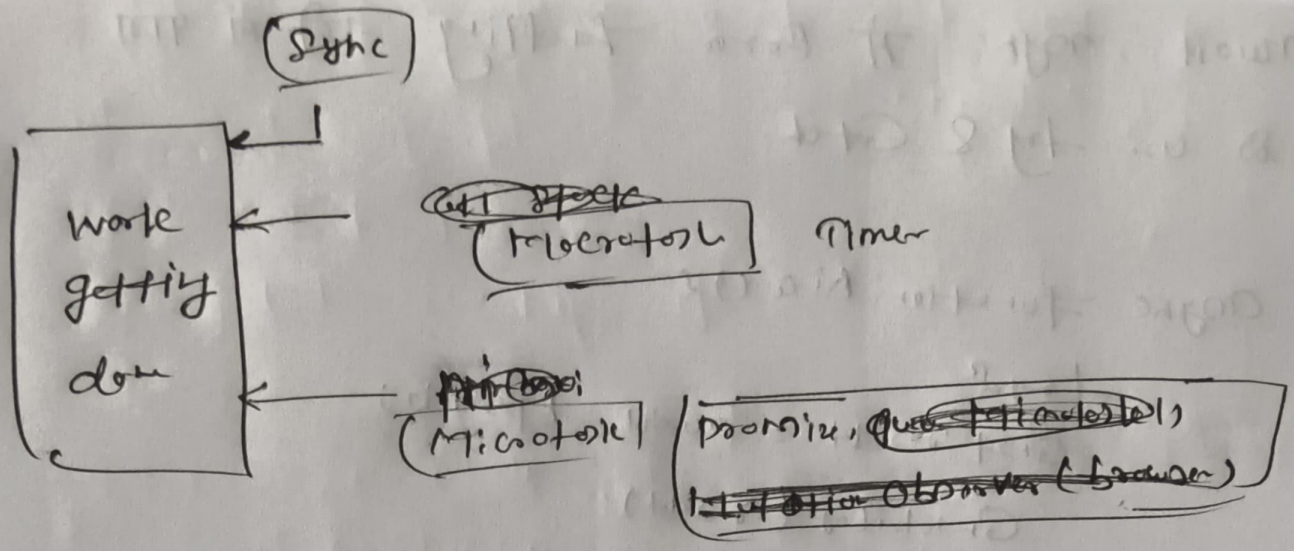
```
Console.log ("Swatik")
promise.resolve ("resolved value").then ((v) => {
        Console.log ("MicroTask", v);
    });
Console.log ("Avishek");
```

↓

Swatik

Avishek

microtask resolved value



→ Synchronous task ki priority too high

→ Promise ki priority

→ timer ki priority

```
function boilWater(ms) {
    return new Promise((res, rej) => {
        console.log("Start to boil water")
        if (typeof ms !== "number" || ms < 0) {
            reject(new Error("ms must be in number
                and greater than zero"));
        }
        setTimeout(() => {
            res("Ubal gya");
        })
    })
}
```

```
boilWater(200)
    .then((msg) => Console.log("Resolved: ", msg))
    .catch((err) => Console.log("Rejected: ", err.message));


function grindLeaves() {
    return Promise.resolve("leaves ground")
}

function steepTea(time) {
    return new Promise((res) => {
        setTimeout(() => res("Steeped tea"), time);
    }, time);

function addSugar(spoon) {
    return `Added ${spoon} sugar`;
}

grindLeaves()
    .then((value
```